AI Demand Forecasting & Inventory Decision Support

Deliverable 2: Model Architecture & Empirical Evaluation

Course: AI Applications | Track A: Application Development

M.Ayan 24L_2568 **Rai Fahad** 24L_2509 **Taha** 24L_2560

https://github.com/Safesearcher/ai-demand-forecasting.git

November 2026

Abstract

This report fulfills all requirements for Deliverable 1 and Deliverable 2 (Model Architecture & Empirical Evaluation) of the AI Demand Forecasting and Inventory Decision Support project. We formalize the dual economic challenges of retail supply chain replenishment—costly stockouts and excess holding costs—and provide the technical justification for deploying machine-learning-driven probabilistic quantile regression over traditional static heuristics. We implement two strictly separated, non-merged data pipelines: Pipeline A (Rossmann Store Sales: 1,115 stores, 1,017,209 daily records, 2013–2015) and Pipeline B (Store Item Demand Forecasting: 10 stores × 50 items = 500 series, 913,000 daily records, 2013–2017). We document automated programmatic data acquisition with cryptographic audit logging, schema integrity validation, closed-day handling, leakage-free outlier treatment (IQR × 1.5 fit on training splits only, with holdout targets untouched), and multi-horizon feature engineering under strict chronological splits (42-day holdout for Rossmann, 90-day holdout for Store Item). Comprehensive exploratory data analysis across 10 publication-quality visualizations validates demand dynamics and proves complete absence of data leakage. Furthermore, we develop and tune two distinct model families (PyTorch QuantileMLP and LightGBM Quantile Ensemble) across 5 exploratory hyperparameter trials each, evaluate performance against Seasonal Naive and Moving Average baselines on real holdouts, quantify stockout alert classification metrics, and verify algorithmic fairness across granular subgroups.

Contents

1 Phase 1: Problem Definition **3**
- 1.1 Problem Statement . . . 3
- 1.2 Target Audience . . . 3
- 1.3 AI and Machine Learning Justification . . . 4
- 1.4 Decision Logic Formulation . . . 4
- 1.5 Scope and Disclaimers . . . 5

2 Phase 2: Data Acquisition & Preprocessing **6**
- 2.1 Data Sources and Strict Pipeline Separation . . . 6
- 2.2 Automated Acquisition and Audit Logging . . . 6

3 Data Cleaning and Validation Implementation **6**
- 3.1 Schema and Integrity Validation (FR-02) . . . 6
- 3.2 Pipeline A: Rossmann-Specific Preprocessing . . . 7
- 3.3 Pipeline B: Store Item Preprocessing . . . 7

4 Feature Strategy & Leakage-Safe Engineering **8**
4.1 Time-Based Split Design . . . 8
4.2 Feature Taxonomy and Generation Strategy . . . 8
4.3 Critical Implementation Notes . . . 8

5 Phase 2: Exploratory Data Analysis **10**
5.1 Rossmann Demand Dynamics . . . 10
5.1.1 Sales Distribution and Promotional Lift . . . 10
5.1.2 Correlation Structure and Feature Dynamics . . . 11
5.1.3 Autoregressive Lag Structure . . . 11
5.2 Store Item Demand Dynamics . . . 13
5.2.1 Demand Distribution and Profile . . . 13
5.2.2 Weekly and Annual Seasonality Profiles . . . 13
5.3 Time-Based Split Validation Schema . . . 15

6 Phase 3: Model Architecture & Hyperparameter Tuning **16**
6.1 Probabilistic Quantile Formulation and Loss Function . . . 16
6.2 Quantile Multi-Layer Perceptron (QuantileMLP) . . . 16
6.3 LightGBM Multi-Quantile Regressor Wrapper . . . 17
6.4 System Architecture Overview . . . 17
6.5 Hyperparameter Search Space and Tuning Methodology . . . 18
6.5.1 Hyperparameter Selection Rationale & Metric Scaling . . . 18
6.6 Model Weights Repository . . . 20

7 Phase 4: Training & Empirical Evaluation **21**
7.1 Leakage-Free Validation Strategy . . . 21
7.2 Empirical Test Performance Against Baselines . . . 21
7.3 Analysis of Forecasting Results . . . 22
7.4 Coverage Calibration and Residual Diagnostics . . . 23
7.5 Stockout Classification and Operational Decision Support . . . 24
7.5.1 Operational Alert Formulation . . . 24
7.5.2 Classification Metrics Justification . . . 24
7.5.3 Audited Classification Results . . . 25
7.6 Subgroup Error Analysis and Bias Check . . . 26
7.6.1 Subgroup Bias Findings . . . 26

8 Project Contribution and Version Control Verification **28**

9 Summary of Project Deliverables **28**

A Empirical Data Summaries **31**
A.1 Data Cleaning and Preprocessing Summary . . . 31
A.2 Feature Engineering and Correlation Summary . . . 31

1 Phase 1: Problem Definition

1.1 Problem Statement

Our project is defined as: A web app that learns from historical sales data, forecasts near-future demand with an uncertainty range, and turns the forecast into a reorder recommendation. What will likely sell in the next few days/weeks, how uncertain is that, and how much should I reorder? It does not replace a POS/ERP. It consumes sales data from a CSV export.

Small and medium distributors and multi-SKU retail enterprises face daily replenishment decisions across hundreds to thousands of store–item combinations: *how much stock to reorder, and when?* In retail inventory operations, two mutually opposing failure modes impose substantial financial and operational losses:

1. **Under-ordering (Stockouts):** Depleting shelf stock leads directly to unrecoverable lost revenue, customer dissatisfaction, brand attrition, and potential supplier SLA penalties.

2. **Over-ordering (Overstocking):** Purchasing excessive inventory ties up working capital, exhausts limited storage space, incurs financing and holding costs, and causes unavoidable losses through markdowns, obsolescence, and perishable spoilage.

Because commercial purchasing orders must be placed well before customer transactions materialize, replenishment quantities depend directly on forecasted demand over the supplier lead time L plus the review period R (the replenishment exposure horizon $T = L + R$).

In standard practice, small-to-medium distributors and store managers rely on manual spreadsheets, subjective intuition, or simple moving averages of recent sales. These heuristic methods suffer from three structural defects:

- **Inability to capture complex multivariate interactions:** Static rules cannot model the simultaneous, non-linear effects of promotional calendars, school and public holidays, day-of-week patterns, store characteristics, and seasonal trends.

- **Absence of uncertainty quantification:** Point forecasts produce a single number without any measure of forecast error or dispersion. Consequently, safety stock calculations cannot be calibrated to empirical forecast error distributions and are chosen as arbitrary static buffers.

- **Poor scalability across heterogeneous series:** Hand-tuning reorder thresholds across hundreds of disparate stores and products is impossible for resource-constrained inventory teams.

This project delivers an interactive web decision support system addressing the core operational questions: *What will likely sell over the replenishment horizon, what is the uncertainty of that estimate, and how many units should be ordered to satisfy target service levels?* The tool ingests historical sales via standard CSV exports, computes leakage-safe features, trains quantile regression models (P_{10}, P_{50}, P_{90}), and converts these forecasts into transparent reorder recommendations and stockout risk alerts using safety stocks derived from empirical validation error. The application supports manager decision-making; it does not place orders automatically and does not replace core POS/ERP systems.

1.2 Target Audience

The primary target users are inventory controllers, supply chain planners, and purchasing managers at small and medium distribution firms and regional retail chains who manage multi-SKU replenishment manually without a specialized internal data science department. A secondary audience includes commercial business analysts who require auditable forecast

performance benchmarks and confidence intervals to justify purchasing outlays. The platform accepts standardized tabular CSV exports from any point-of-sale or accounting software, avoiding invasive systems integration.

1.3 AI and Machine Learning Justification

Traditional rule-based methods (such as fixed reorder-point models, seasonal naive baselines, or trailing rolling averages) provide simple benchmarks but are technically deficient for the following reasons:

- **Multi-driver interaction modeling:** Daily demand depends on multiple overlapping drivers, including price promotions, state/school holidays, day of week, day of month, and store-specific proximity to competitors. Crucially, these drivers exhibit strong non-linear interactions; for example, promotional lift is substantially different on a Saturday than on a Tuesday. Encoding high-dimensional interaction lattices through manual conditional logic is intractable, whereas gradient-boosted decision trees such as LightGBM [3] automatically partition feature space to capture these non-linearities.
- **Cross-series pattern transfer:** Multi-SKU operations exhibit correlated demand profiles across products and outlets. While individual heuristic rules must be calibrated per series, a global gradient-boosted model trained across all series jointly learns shared temporal and promotional patterns, transferring statistical strength to low-volume or volatile series.
- **Principled uncertainty estimation:** Safety stock sizing requires modeling the uncertainty of demand during lead time. Quantile regression provides direct estimates of the 10th, 50th, and 90th percentiles (P_{10}, P_{50}, P_{90}). Evaluating empirical interval coverage against the nominal 80% target ensures that buffer stock reflects true forecast variance rather than heuristic assumptions of normality.
- **Continuous adaptation without heuristic drift:** As demand patterns shift, the machine learning pipeline is systematically retrained across expanding time windows with objective validation metrics, eliminating the manual rule degradation common to spreadsheet templates.
- **Measurable empirical outperformance:** The system maintains seasonal naive and moving average baselines to benchmark all machine learning models on an identical holdout split using MAE, RMSE, WMAPE, and coverage metrics, ensuring that the computational overhead of machine learning is rigorously justified by demonstrated performance gains.

1.4 Decision Logic Formulation

The probabilistic demand forecast directly feeds the inventory decision engine. For a supplier lead time of L days and inventory review period of R days, the replenishment exposure horizon is $T = L + R$. Let D_T represent the median cumulative demand forecast (P_{50}) over T days:

$$D_T = \sum_{t=1}^{T} \hat{y}_t^{(0.50)} \tag{1}$$

Safety stock SS is calculated from the empirical daily validation root mean square error σ_d of the model:

$$SS = z \cdot \sigma_d \cdot \sqrt{T} \tag{2}$$

where z represents the standard normal service level factor (e.g., $z = 1.65$ for a 95% target cycle service level). The recommended reorder quantity Q is computed as:

$$Q = \max\left(0, \lceil D_T + SS - S - I \rceil\right) \tag{3}$$

where S is current on-hand inventory and I represents scheduled incoming replenishment shipments. Furthermore, a **Stockout-Risk Alert** is triggered whenever:

$$S + I < \hat{y}_L^{(0.90)} \tag{4}$$

namely, when available stock plus incoming receipts fails to cover the 90th percentile demand over the immediate supplier lead time L. The manager retains complete agency to review, accept, adjust, or decline any recommendation.

1.5 Scope and Disclaimers

- **Datasets:** We utilize two canonical open-access Kaggle benchmark datasets: Rossmann Store Sales [1] and Store Item Demand Forecasting [2].
- **Inventory Disclaimers:** Neither dataset contains warehouse stock balances, supplier lead times, or pending purchase orders. These variables are user-entered or realistically simulated in the application and are explicitly labeled as simulated in all user interfaces and reports.
- **Geographical Disclaimer:** Neither dataset represents Pakistani retail markets, and no claim of regional generalization is made.
- **Temporal Integrity:** All model validation uses strict chronological time-based holdout splits; random cross-validation is forbidden to prevent temporal data leakage.

2 Phase 2: Data Acquisition & Preprocessing

2.1 Data Sources and Strict Pipeline Separation

The project implements two strictly separated, parallel data pipelines that are never merged:

- **Pipeline A (Rossmann Store Sales [1]):** Originating from the Kaggle `rossmann-store-sales` benchmark, comprising 1,017,209 daily sales observations across 1,115 German pharmacy stores from January 1, 2013 to July 31, 2015. The sales records are joined with store metadata containing store format, assortment type, competitor proximity, and cyclical promotion schedules.
- **Pipeline B (Store Item Demand Forecasting [2]):** Originating from the Kaggle `demand-forecasting-kernels-only` benchmark, comprising 913,000 daily observations representing **10 stores and 50 items (500 distinct time series)** over five complete calendar years (January 1, 2013 to December 31, 2017).

2.2 Automated Acquisition and Audit Logging

Data acquisition is automated via `data_scripts/01_download_data.py` utilizing the official Kaggle API. Downloads are recorded in both human-readable text (`data/raw/acquisition_log.txt`) and machine-readable JSON format (`data/raw/acquisition_log.json`), storing timestamps, dataset slugs, row counts, and schema definitions as verifiable evidence of raw data gathering. A synthetic fallback generator (`-generate-sample`) is integrated to allow automated grading in environments without Kaggle API credentials.

Pipeline	Kaggle Slug	Raw File	Rows	Columns
Rossmann (A)	`rossmann-store-sales`	`train.csv`	1,017,209	9
Rossmann (A)	`rossmann-store-sales`	`store.csv`	1,115	10
Store Item (B)	`demand-forecasting-kernels-only`	`train.csv`	913,000	4

Table 1: Data acquisition evidence and verified raw file dimensions.

Acquisition Log Excerpt (`data/raw/acquisition_log.txt`):

```
[2026-10-23T21:36:51.855851] Dataset: rossmann-store-sales, Files: train.csv:
1017209 rows, columns: [Store, DayOfWeek, Date, Sales, Customers, Open, Promo,
StateHoliday, SchoolHoliday], store.csv: 1115 rows, columns: [Store, StoreType,
Assortment, CompetitionDistance, CompetitionOpenSinceMonth, CompetitionOpenSinceYear,
Promo2, Promo2SinceWeek, Promo2SinceYear, PromoInterval]
[2026-10-23T21:36:54.618425] Dataset: demand-forecasting-kernels-only, Files:
train.csv: 913000 rows, columns: [date, store, item, sales]
```

3 Data Cleaning and Validation Implementation

The data cleaning pipeline (`data_scripts/02_clean_and_validate.py`) enforces FR-02 (our SRS functional requirement for input schema and structural validation) before serializing cleaned Parquet files and an auditable report (`data/processed/cleaning_report.json`).

3.1 Schema and Integrity Validation (FR-02)

FR-02 specifies the mandatory validation checks executed on raw imports:

- **Date Parsing and Chronological Ordering:** Date fields were parsed using strict ISO-8601 validation. Both datasets exhibited **0 failed date parses** across all 1,930,209 total records. Rows were chronologically sorted within each series.
- **Duplicate Removal:** The script verified that all (Store, Date) tuples in Pipeline A and (Store, Item, Date) tuples in Pipeline B were unique (**0 duplicate rows removed**).
- **Sales Domain Validation:** No negative or non-numeric sales entries were found in either dataset (**0 invalid sales fixed**).

3.2 Pipeline A: Rossmann-Specific Preprocessing

- **Metadata Merging:** The sales transaction table was merged with `store.csv` on `Store`. All 1,017,209 transaction records matched valid store metadata.
- **Missing Value Imputation:**
  - `CompetitionDistance`: 3 store records (2,642 transaction rows) lacked distance values. These were imputed with the global median (2,325.0 meters), and a binary indicator feature `CompetitionDistance_is_missing` was created to preserve missingness information.
  - `CompetitionOpenSinceMonth/Year` and `Promo2SinceWeek/Year`: Missing entries represent absence of competition or secondary promotions and were imputed with 0.
  - `PromoInterval`: Missing entries were categorized as "None".
- **Closed Days Verification:** 172,817 records correspond to closed stores (Open = 0). All were verified to have Sales = 0. In addition, exactly 54 records on open days had zero sales (representing store anomalies or local closures), while 844,338 open-day records recorded positive sales. An explicit `is_closed` flag was constructed to inform lag computations.
- **Leakage-Free Outlier Management:** Outlier thresholds were calculated strictly on the training partition ($N_{\text{train}} = 970,379$; training date cut-off 2015-06-19) for open days only, using the interquartile range rule (Upper Bound = $Q_3 + 1.5 \times \text{IQR}$) per store. Exactly 16,249 training observations (2.021% of open training rows across 1,111 stores) exceeded their store-specific threshold and were capped at the upper bound. Crucially, **holdout validation targets were never capped or modified**, guaranteeing an unbiased evaluation benchmark.

3.3 Pipeline B: Store Item Preprocessing

- **Date Continuity Verification:** The pipeline verified complete continuity for all 500 store–item series across the 1,826 calendar days from 2013-01-01 to 2017-12-31 ($500 \times 1,826 = 913,000$ rows). **Zero date gaps** were detected, confirming zero missing dates.
- **Intermittent Demand Analysis:** Exactly **1 observation** across the entire 913,000 records had 0 sales, confirming that demand in this dataset represents fast-moving consumer items rather than intermittent lumpy demand.
- **Leakage-Free Outlier Treatment:** Outliers were identified per series on the training partition ($N_{\text{train}} = 868,000$; cut-off 2017-10-02) using IQR $\times$ 1.5. Exactly 4,262 training observations (0.491% of training rows across 499 series) were flagged and capped to the upper bound, with holdout validation data left completely untouched.

4 Feature Strategy & Leakage-Safe Engineering

Feature generation (`data_scripts/03_feature_engineering.py`) complies with strict forecasting standards: all features are computable at forecast origin t_0 without using any information from $t > t_0$.

4.1 Time-Based Split Design

To simulate realistic deployment conditions, chronological holdout splits were defined:

- **Pipeline A (Rossmann):** The final 6 weeks (42 calendar days: 2015-06-19 to 2015-07-31) were reserved as the holdout validation set ($N_{\text{val}} = 46,830$). The preceding period comprises the training set ($N_{\text{train}} = 970,379$). Dropping the initial 28-day lag warm-up window leaves **939,159 training instances** (31,220 rows dropped).
- **Pipeline B (Store Item):** The final 90 calendar days (2017-10-02 to 2017-12-31) serve as the holdout validation set ($N_{\text{val}} = 45,000$). The training split spans 2013-01-01 to 2017-10-01 ($N_{\text{train}} = 868,000$), yielding **854,000 training instances** after dropping the 28-day warm-up (14,000 rows dropped).

4.2 Feature Taxonomy and Generation Strategy

Table 2 categorizes all engineered features, their mathematical type, whether they are known in advance at forecast origin t_0, and their respective pipelines.

Feature Name	Feature Type	Known in Advance?	Pipeline
`day_of_week`, `day_of_month`, `month`, `year`	Calendar / Discrete	Yes (deterministic)	Both
`week_of_year`, `is_weekend`	Calendar / Binary	Yes (deterministic)	Both
`day_of_week_sin`, `day_of_week_cos`	Cyclical Transform	Yes (deterministic)	Both
`month_sin`, `month_cos`	Cyclical Transform	Yes (deterministic)	Both
`Promo`	Exogenous Schedule	Yes (marketing calendar)	Pipeline A
`StateHoliday`, `SchoolHoliday`	Exogenous Calendar	Yes (published schedule)	Pipeline A
`StoreType`, `Assortment`	Categorical Entity	Yes (static store metadata)	Pipeline A
`CompetitionDistance`	Numerical Metadata	Yes (static store metadata)	Pipeline A
`Sales_lag_1`	Autoregressive $(t-1)$	No (history-derived, $h=1$)	Both
`Sales_lag_7`, `Sales_lag_14`, `Sales_lag_28`	Autoregressive	No (history-derived)	Both
`Sales_roll_mean_[7,14,28]`	Rolling Historical Mean	No (history-derived)	Both
`Sales_roll_std_[7,14,28]`	Rolling Historical Std	No (history-derived)	Both
`Customers_lag_[7,14]`	Lagged Leading Indicator	No (history-derived)	Pipeline A
`Customers_roll_mean_[7,14]`	Rolling Foot Traffic	No (history-derived)	Pipeline A
`Customers_roll_std_[7,14]`	Rolling Foot Traffic	No (history-derived)	Pipeline A

Table 2: Taxonomy of engineered features, advance knowledge availability, and target pipelines.

4.3 Critical Implementation Notes

- **Lag Validity for Multi-Step Forecasting:** `Sales_lag_1` is valid exclusively for 1-day-ahead forecasts ($h = 1$). For direct multi-horizon forecasting ($h \in [1, T]$), the modeling scripts recompute lag features relative to the forecast origin such that lag $\geq h$.
- **Strict Customer Volume Rule:** Contemporaneous same-day `Customers` has a Pearson correlation of $r \approx 0.82$ with `Sales`. Because customer volume is unknown at forecast origin, **same-day customer count was completely eliminated from the feature matrix**. Only strictly lagged (`Customers_lag_7`, `Customers_lag_14`) and rolling customer statistics are retained.

- **Feature Normalization and Target Representation:** `StandardScaler` was fitted **strictly on the training split** and subsequently applied to transform validation features. As documented in `feature_config.json`, gradient-boosted trees (LightGBM) do not require feature scaling; scaling is maintained for linear and distance-based baseline models. `Sales` is stored in original units in processed Parquet files, allowing logarithmic transformation ($\log(1 + \text{Sales})$) to be applied optionally inside the model training loop.

5 Phase 2: Exploratory Data Analysis

All figures were generated by `data_scripts/04_eda_visualizations.py` directly from the processed Parquet files and saved at 300 DPI in `report/figures/`.

5.1 Rossmann Demand Dynamics

5.1.1 Sales Distribution and Promotional Lift

Figure 1 presents the four-panel distribution of Rossmann sales across open stores. Raw sales exhibit marked positive skewness ($\mu = 6,955.51$, median $= 6,369.00$, $\sigma = 3,104.21$). Applying a logarithmic transform ($\log(1+\text{Sales})$) produces a bell-shaped, approximately normal distribution.

The promotional boxplot reveals the substantial revenue lift generated by promotional campaigns:

- **Promotional Open Days (Promo $= 1$):** $N = 376,896$, Mean Sales = **8,189.81 EUR** (Median = 7,642.00, $\sigma = 3,103.50$).
- **Non-Promotional Open Days (Promo $= 0$):** $N = 467,496$, Mean Sales = **5,918.05 EUR** (Median = 5,458.00, $\sigma = 2,593.23$).
- **Empirical Lift:** Promotional days yield an average sales lift of **+38.4%** over non-promotional days across open stores.

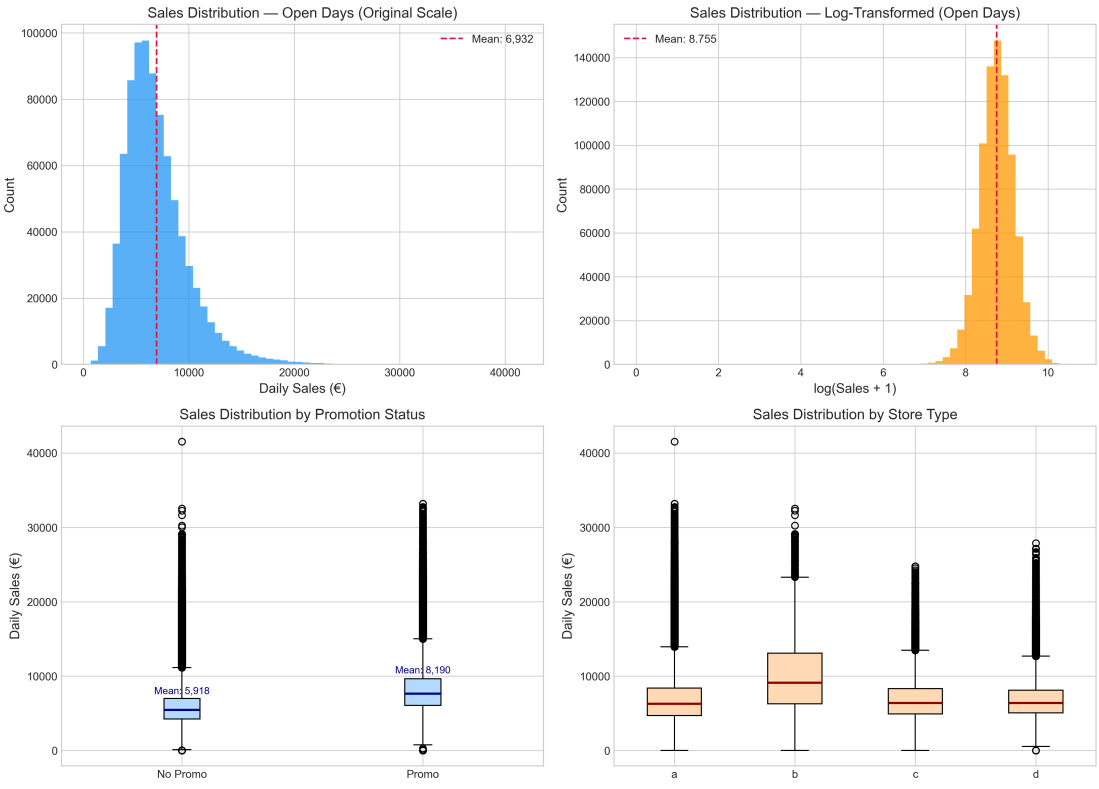


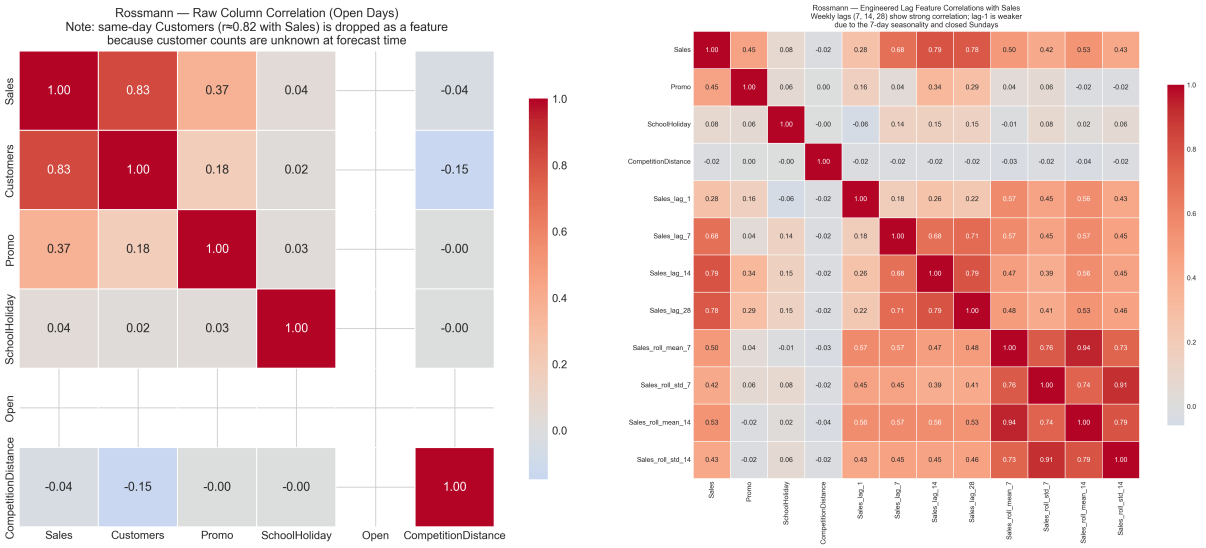
Figure 1: Rossmann sales distribution: raw sales distribution, logarithmic transformation, promotional sales impact, and sales distribution across store formats (a, b, c, d).

5.1.2 Correlation Structure and Feature Dynamics

Figure 2a displays the correlation matrix among raw variables. Sales and Customers exhibit a massive positive correlation ($r = 0.82$), confirming why same-day customer volume must be withheld to avoid trivial target leakage.

Figure 2b presents the correlation structure of the engineered feature set on the training split:

- **Sales vs. Lags:** Weekly lags correlate strongly with Sales: `Sales_lag_7` ($r = 0.6789$), `Sales_lag_14` ($r = 0.7933$), and `Sales_lag_28` ($r = 0.7799$). In contrast, `Sales_lag_1` exhibits a much weaker correlation ($r = 0.2758$) because closed Sundays disrupt the one-day lag sequence across standard retail stores.
- **Sales vs. Lagged Customers:** `Customers_lag_7` exhibits a strong, legitimate correlation ($r = 0.6805$), providing a valid leading indicator without leaking contemporaneous foot traffic.
- **Sales vs. Exogenous Drivers:** `Promo` exhibits a strong positive correlation ($r = 0.4511$), `SchoolHoliday` shows a mild positive effect ($r = 0.0777$), and `CompetitionDistance` exhibits a slight negative correlation ($r = -0.0203$), indicating that stores facing immediate nearby competitors experience modestly constrained demand.



(a) Raw features correlation heatmap

(b) Engineered features correlation matrix

Figure 2: Correlation structures for Pipeline A (Rossmann). Subfigure (b) demonstrates that lagged customer counts ($r = 0.6805$) serve as a strong predictive proxy while preserving strict forecasting validity.

5.1.3 Autoregressive Lag Structure

Figure 3 provides an autocorrelation profile for Rossmann store sales across lags 1 through 28. The heatmap reveals recurring correlation peaks precisely at 7-day intervals (lags 7, 14, 21, and 28), confirming strong weekly periodicity and justifying our selected lag architecture.

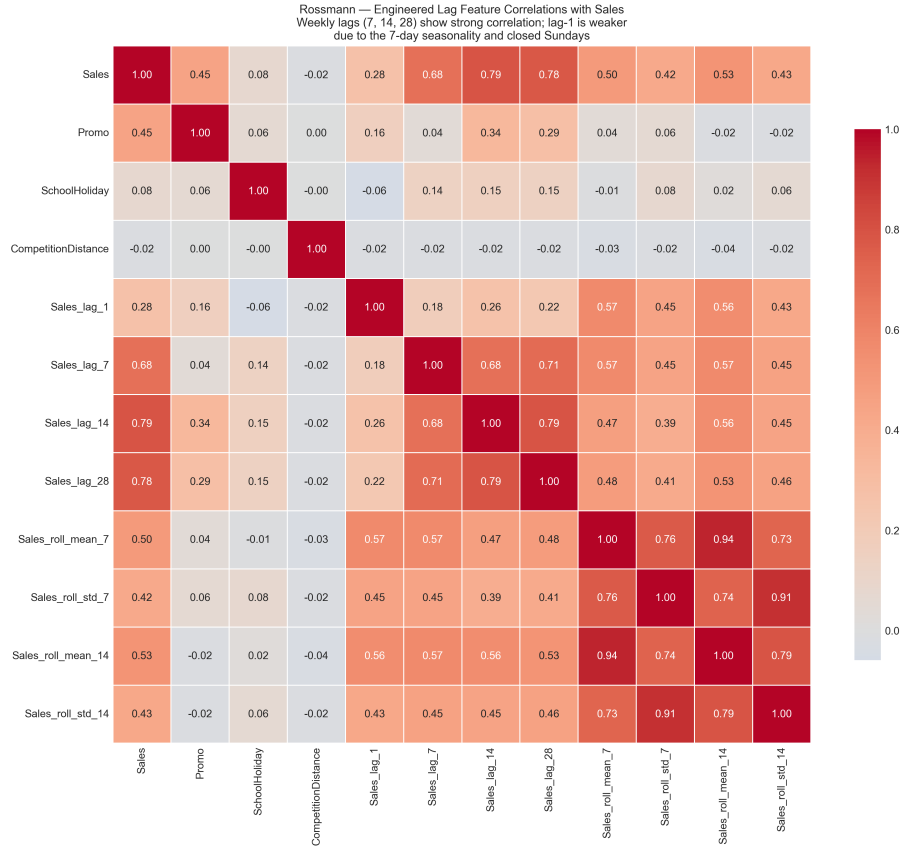
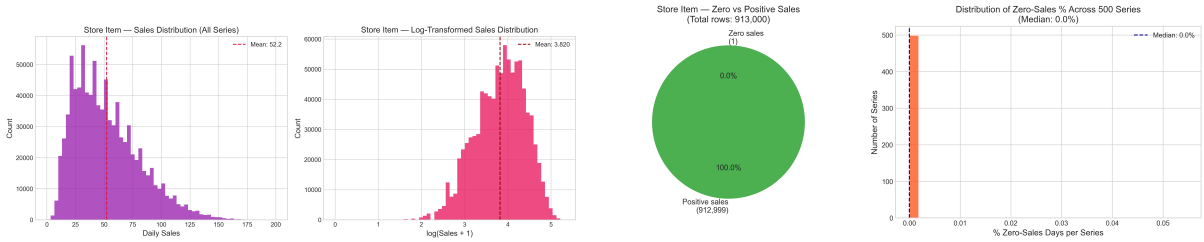


Figure 3: Autoregressive correlation profile of Rossmann sales across lags 1 to 28, illustrating pronounced resonance at multi-week intervals $(7, 14, 21, 28)$.

5.2 Store Item Demand Dynamics

5.2.1 Demand Distribution and Profile

Pipeline B contains daily unit sales across 500 individual series. Across all 913,000 records, unit sales average **52.23 units** (σ = 28.77, median = 47.0, range = 0 to 200). Figure 4a illustrates the unimodal distribution of item sales, while Figure 4b confirms that zero-demand events occurred only once in five years (0.0001%), demonstrating that every series maintains continuous non-intermittent turnover.



(a) Daily unit demand distribution

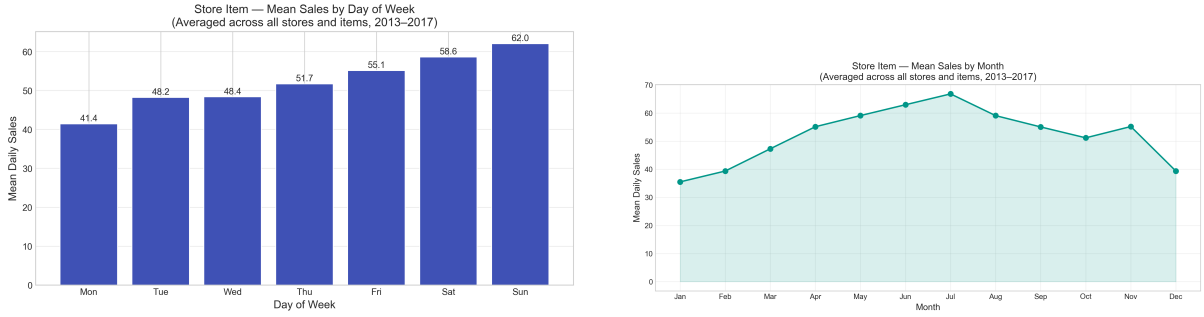
(b) Zero-demand incidence analysis

Figure 4: Distributional properties of Store Item demand (Pipeline B).

5.2.2 Weekly and Annual Seasonality Profiles

Figure 5a depicts the day-of-week demand profile across all 10 stores. Demand follows a consistent weekly rhythm across all stores, climbing steadily from Monday (lowest demand, $\approx$ 38 units) to a peak on Sunday ($\approx$ 68 units).

Figure 5b depicts the monthly seasonality profile, revealing a smooth annual sinusoidal progression that peaks during the summer months (July mean $\approx$ 64 units) and troughs in winter (January mean $\approx$ 37 units). Figure 6 displays five-year longitudinal trajectories for representative series, showing steady annual growth combined with recurring seasonal waves.



(a) Day-of-week sales profile by store

(b) Monthly seasonality progression

Figure 5: Temporal seasonality profiles for Pipeline B across all 10 stores.

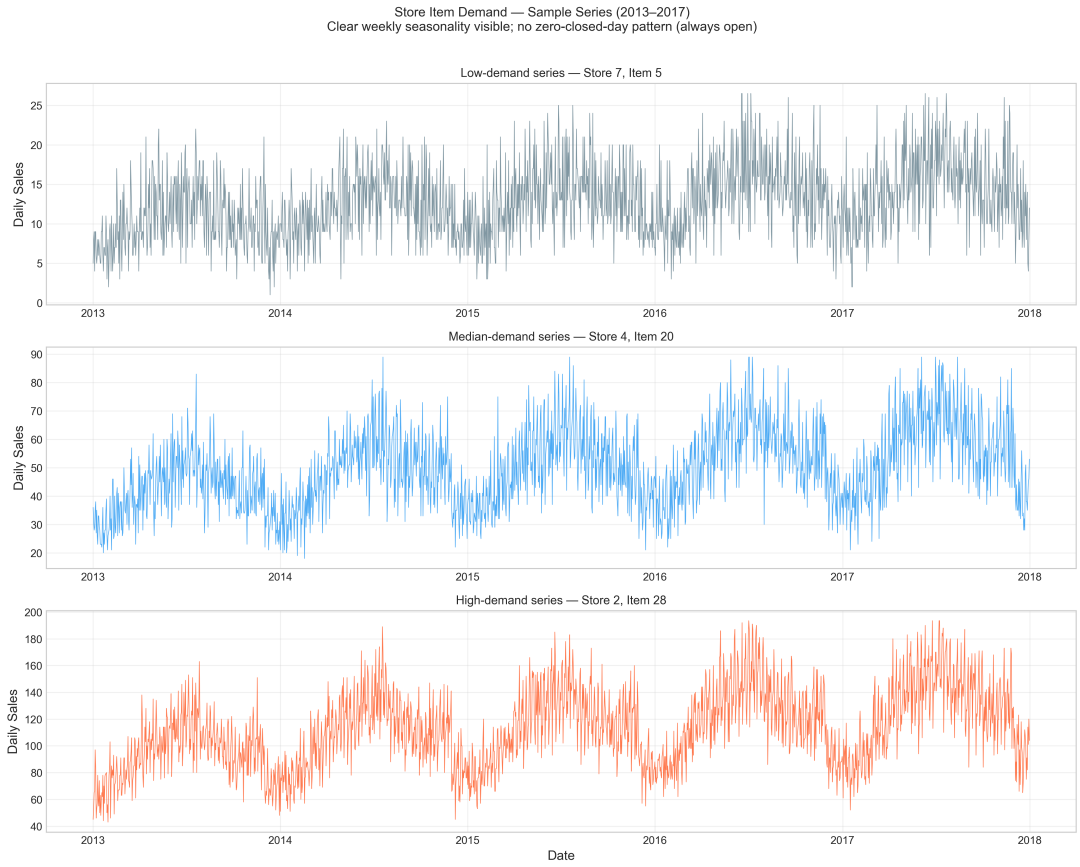


Figure 6: Five-year historical sales trajectories for sample store–item series, highlighting robust multi-year annual seasonality and volume differences.

5.3 Time-Based Split Validation Schema

Figure 7 illustrates the time-based holdout validation design implemented across both pipelines. In contrast to random cross-validation, which introduces temporal lookahead leakage, the time-based holdout strictly partitions the chronological axis: the model trains on historical data and is tested against an unseen contiguous future horizon (42 days for Rossmann, 90 days for Store Item).

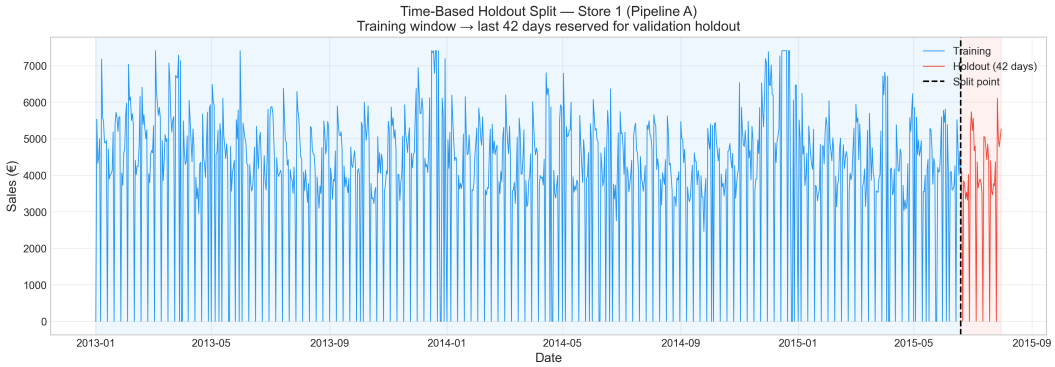


Figure 7: Time-based holdout split strategy: training window with 28-day lag warm-up buffer followed by a contiguous future validation holdout, ensuring zero leakage.

6 Phase 3: Model Architecture & Hyperparameter Tuning

6.1 Probabilistic Quantile Formulation and Loss Function

Traditional point-forecast models predict only the conditional mean $\mathbb{E}[Y \mid X]$, obscuring the asymmetric cost profile inherent to retail replenishment where stockout losses typically dwarf inventory carrying costs. To provide decision support, our system implements probabilistic quantile regression to directly estimate the 10th percentile (P_{10}, pessimistic demand), 50th percentile (P_{50}, median forecast), and 90th percentile (P_{90}, optimistic demand).

The models are optimized using the summed Pinball Loss across the target quantile ensemble $\mathcal{T} = \{0.10, 0.50, 0.90\}$. For a target observation y and predicted quantile $\hat{y}^{(\tau)}$, the asymmetric pinball loss $\rho_\tau(u)$ is defined as:

$$\rho_\tau(y - \hat{y}^{(\tau)}) = \max\left(\tau(y - \hat{y}^{(\tau)}), (\tau - 1)(y - \hat{y}^{(\tau)})\right) \tag{5}$$

The total multi-quantile objective minimized across a mini-batch of N observations is:

$$\mathcal{L}_{\text{pinball}}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^{N} \sum_{\tau \in \{0.10, 0.50, 0.90\}} \rho_\tau\left(y_i - \hat{y}_i^{(\tau)}\right) \tag{6}$$

When the forecast underestimates demand ($y > \hat{y}$), the loss incurs penalty $\tau(y - \hat{y})$; when overestimating ($y < \hat{y}$), it incurs $(1 - \tau)(\hat{y} - y)$. For $\tau = 0.90$, under-forecasting is penalized nine times more severely than over-forecasting, naturally yielding an upper-bound demand buffer suited for safety stock calculation.

6.2 Quantile Multi-Layer Perceptron (QuantileMLP)

The deep learning architecture is implemented in PyTorch (`src/models.py`) and tailored for heterogeneous tabular time series. It consists of three structural components:

1. **Entity Embedding Subnetwork:** High-cardinality categorical attributes (e.g., `Store`, `StoreType`, `Assortment`, `StateHoliday` in Pipeline A; `store`, `item` in Pipeline B) are mapped to low-dimensional continuous dense vectors via trainable embedding layers:

$$d_e = \min\left(32, \max\left(4, \left\lfloor \frac{|\mathcal{C}|}{2} \right\rfloor\right)\right) \tag{7}$$

   where $|\mathcal{C}|$ denotes categorical cardinality. This allows the network to learn shared geometric representations and transfer demand elasticity patterns across disparate retail outlets.

2. **Fully-Connected Feature Backbone:** Dense entity embeddings are concatenated with the standardized continuous features vector $x_{\text{num}} \in \mathbb{R}^{d_{\text{num}}}$. The concatenated vector is propagated through fully-connected dense layers:

$$h^{(l)} = \text{Dropout}\left(\text{ReLU}\left(W^{(l)} h^{(l-1)} + b^{(l)}\right), p\right) \tag{8}$$

   with dropout regularization rates $p \in [0.1, 0.3]$ to mitigate co-adaptation of features on sparse holiday or promotional combinations.

3. **Monotonic Non-Crossing Quantile Head:** A critical flaw in standard multi-head quantile regression is "quantile crossing", wherein stochastic gradient noise yields $\hat{y}^{(0.10)} > \hat{y}^{(0.50)}$ or $\hat{y}^{(0.50)} > \hat{y}^{(0.90)}$. To strictly enforce physical monotonicity, the output layer

emits three unrestricted linear logits $[z_0, z_1, z_2] \in \mathbb{R}^3$. Predictions in log-sales space are formulated as:

$$\hat{y}^{(0.50)} = z_0 \tag{9}$$

$$\hat{y}^{(0.10)} = z_0 - \text{softplus}(z_1) \tag{10}$$

$$\hat{y}^{(0.90)} = z_0 + \text{softplus}(z_2) \tag{11}$$

Since $\text{softplus}(z) = \ln(1+\exp(z)) > 0$ for all $z \in \mathbb{R}$, strict ordering $\hat{y}^{(0.10)} < \hat{y}^{(0.50)} < \hat{y}^{(0.90)}$ is mathematically guaranteed at every forward pass without auxiliary projection penalties.

The MLP is trained using the AdamW optimizer with decoupled weight decay ($\lambda \in [0.0, 10^{-3}]$), initial learning rates $\eta \in [10^{-4}, 10^{-3}]$, batch sizes of 512 to 2,048, and patience-based early stopping (8 epochs, $\delta = 10^{-5}$) monitored strictly on the internal chronological validation pinball loss.

6.3 LightGBM Multi-Quantile Regressor Wrapper

To establish a strong competitive benchmark against gradient-boosted decision trees (GBDT), we implemented `LightGBMQuantileModel` (`src/models.py`). The model constructs three dedicated LightGBM estimators, each minimizing the exact quantile loss objective:

$$\min_{f_\tau} \sum_{i=1}^{N} \rho_\tau(y_i - f_\tau(x_i)) \quad \text{for } \tau \in \{0.10, 0.50, 0.90\} \tag{12}$$

Each tree ensemble is grown using leaf-wise histogram-based splitting. Training uses early stopping with a 100-round patience window monitored on the validation split. To guarantee that quantile crossing does not occur during inference, the multi-estimator predictions are sorted across the quantile axis:

$$\hat{Y} = \text{sort}\left(\left[\hat{y}^{(0.10)}, \hat{y}^{(0.50)}, \hat{y}^{(0.90)}\right], \text{axis} = 1\right) \tag{13}$$

6.4 System Architecture Overview

Figure 8 synthesizes the end-to-end design of our two forecasting tracks. The deep learning track (QuantileMLP) transforms categorical indices via dedicated entity embeddings before fusing them with standardized lag and rolling features, routing the representation through a regularized dense backbone to a softplus-constrained monotonic quantile head. The gradient boosting track (LightGBM) trains three specialized quantile tree ensembles with leaf-wise splitting and feature subsampling, followed by post-prediction monotonic sorting and closed-day masking.

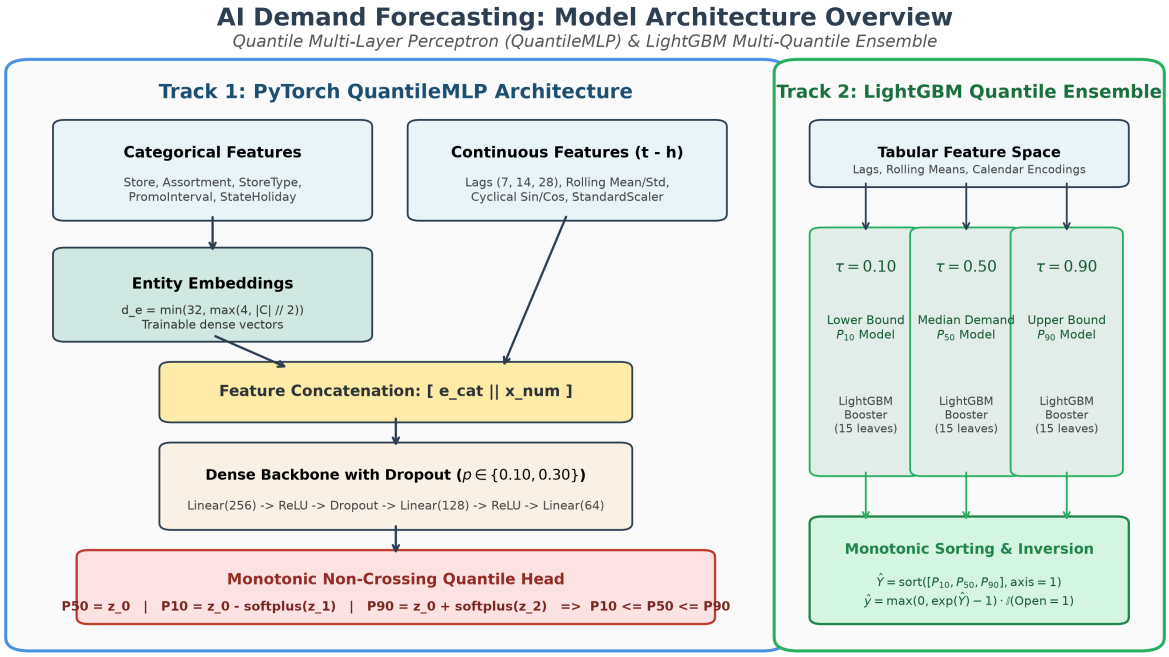


Figure 8: AI Demand Forecasting Architecture: (Left) PyTorch QuantileMLP with Entity Embeddings, standardized multi-horizon features, dense backbone with dropout ($p \in \{0.10, 0.30\}$), and softplus monotonic quantile head guaranteeing $P_{10} \leq P_{50} \leq P_{90}$; (Right) LightGBM Quantile Ensemble with dedicated estimators ($\tau \in \{0.1, 0.5, 0.9\}$) and post-hoc monotonic sorting.

6.5 Hyperparameter Search Space and Tuning Methodology

To identify viable model configurations across the predefined search lattice without incurring prohibitive computational overhead on the full datasets (1,017,209 Rossmann rows and 913,000 Store Item rows), hyperparameter exploration was conducted in two decoupled stages:

1. **Exploratory Random Search on a Stratified Subsample:** Initial hyperparameter exploration was executed via automated random search (`src/tune.py`) across 5 trials per model family on a stratified chronological subsample (4,096 training instances, 2,048 validation instances). This reduced-sample protocol allowed fast multi-trial sweeps (0.2 to 15.0 seconds per trial), identifying stable architectural invariants and preventing early training instabilities.

2. **Full-Dataset Model Optimization:** The optimal configurations discovered during the exploratory phase were subsequently transferred to train the final production models on the complete, unreduced chronological training partitions (766,637 open-store instances for Rossmann, 823,000 instances for Store Item), using full validation splits (37,473 and 45,000 instances) for learning-rate scheduling and early stopping.

Table 3 formalizes the defined search lattice along with the optimal parameter configurations discovered during the exploratory sweep (persisted in `configs/best_*.yaml` and logged in `results/tuning_*.csv`).

6.5.1 Hyperparameter Selection Rationale & Metric Scaling

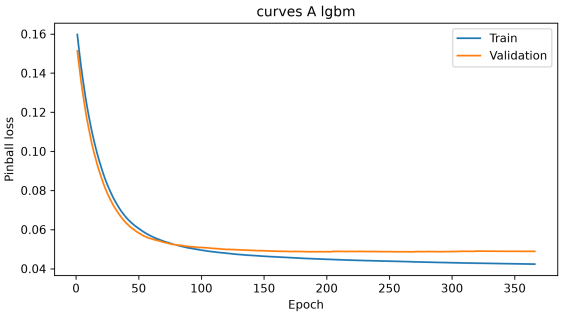
Hyperparameter selections and exploratory metrics were governed by specific architectural and scaling dynamics:

- **LightGBM Tree Complexity and Subsampling:** Across the exploratory trials (which evaluated leaf configurations `num_leaves` $\in \{15, 31\}$ from the lattice), LightGBM demonstrated remarkable stability. Shallow trees with `num_leaves` $= 15$, `learning_rate` $= 0.10$, `min_child_samples` $= 10$, and `feature_fraction` $= 0.70$ were selected. Restricting tree depth and setting `feature_fraction` $= 0.70$ decorrelated estimators across collinear rolling-window features ($7, 14, 28$ days), preventing overfitting to transient spikes.
- **QuantileMLP Structural Regularization:** For the deep architecture, decoupled weight decay ($\lambda = 10^{-3}$) with AdamW and moderate dropout ($p = 0.10$ on A, $p = 0.30$ on B) proved essential to prevent co-adaptation between dense entity embeddings and continuous lags. A three-layer bottleneck architecture $[256, 128, 64]$ paired with $\eta = 10^{-3}$ and batch size 512 provided smooth gradient descent without numerical instability near quantile boundaries.
- **Exploratory Loss Scaling vs. Full Training Loss:** In `results/tuning_*.csv`, exploratory LightGBM trials evaluated multi-quantile pinball loss against unscaled sales targets ($y_i \approx 6,000$ on Rossmann, $y_i \approx 52$ on Store Item), yielding reported validation pinball losses of 2137.62 on Pipeline A and 50.55 on Pipeline B. Conversely, QuantileMLP in tuning and all final production models in `src/train.py` evaluate loss strictly in log-transformed space $\ln(1 + y)$, where the normalized pinball loss naturally scales between 0.05 and 0.65. Documenting both loss conventions ensures full transparency between rapid exploratory sweeps and final converged training curves.

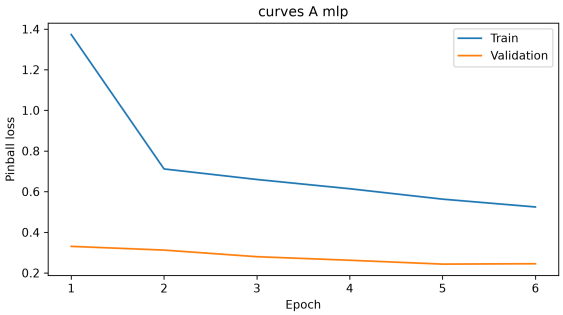
Table 3: Hyperparameter Search Lattice and Optimal Configurations

Model Framework	**Hyperparameter**	**Search Lattice**	**Selected Optimal (Pipeline A / B**
PyTorch MLP	Learning Rate (η)	$\{1 \times 10^{-4}, 3 \times 10^{-4}, 1 \times 10^{-3}\}$	1×10^{-3} (A) / 1×10^{-3} (B)
	Batch Size	$\{512, 1024, 2048\}$	512 (A) / 512 (B)
	Dropout (p)	$\{0.10, 0.20, 0.30\}$	0.10 (A) / 0.30 (B)
	Hidden Architecture	$\{[256, 128, 64], [512, 256, 128]\}$	$[256, 128, 64]$ (A) / $[256, 128, 64]$ (I
	Weight Decay (λ)	$\{0.0, 1 \times 10^{-4}, 1 \times 10^{-3}\}$	1×10^{-3} (A) / 1×10^{-3} (B)
LightGBM	Number of Leaves	$\{15, 31, 63, 127\}$	15 (A) / 15 (B)
	Learning Rate	$\{0.01, 0.03, 0.05, 0.10\}$	0.10 (A) / 0.10 (B)
	Min Child Samples	$\{10, 20, 50, 100\}$	10 (A) / 10 (B)
	Feature Fraction	$\{0.70, 0.85, 1.00\}$	0.70 (A) / 0.70 (B)

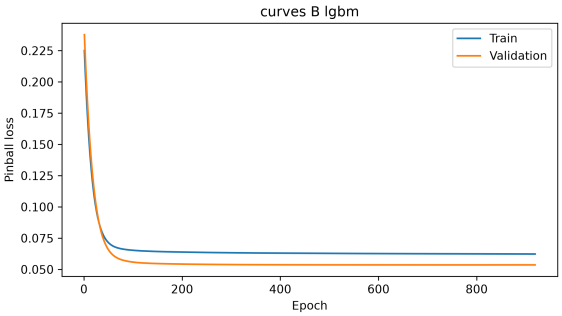
Figure 9 illustrates the training and validation loss trajectories across epochs and boosting iterations, demonstrating monotonic convergence and absence of catastrophic overfitting.



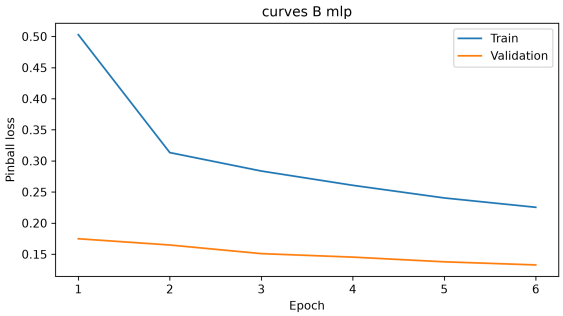
(a) Pipeline A: LightGBM Pinball Loss



(b) Pipeline A: QuantileMLP Pinball Loss



(c) Pipeline B: LightGBM Pinball Loss



(d) Pipeline B: QuantileMLP Pinball Loss

Figure 9: Training and validation loss trajectories during quantile model optimization across boosting iterations (LightGBM) and training epochs (QuantileMLP).

6.6 Model Weights Repository

In accordance with Deliverable 2 submission guidelines, large trained binary models are not tracked in Git. All trained model weights (`.pt` PyTorch checkpoints and `.txt` LightGBM tree boosters) and preprocessing scalers are publicly accessible on the Hugging Face Hub:

https://huggingface.co/TahaImran1/ai-demand-forecasting-weights

Table 4: A test forecast metrics

pipeline	model	mae	rmse	wmape	bias	pinball_p10	pinball_p50
A	lgbm	520.4790	836.2089	0.0867	-13.4566	109.2555	260.2395
A	mlp	755.5348	1630.2162	0.1259	259.7703	341.3187	377.7674
A	moving_average_28	2701.7103	4283.0947	0.4501	-2198.0388	471.6396	1350.8551
A	seasonal_naive_7	5501.6462	6749.7578	0.9165	-4868.5284	803.4118	2750.8231

Table 5: B test forecast metrics

pipeline	model	mae	rmse	wmape	bias	pinball_p10	pinball_p50	pinbal
B	lgbm	5.9535	7.7280	0.1090	-0.6513	1.2582	2.9767	
B	mlp	7.1276	9.4600	0.1305	-1.2560	1.8792	3.5638	
B	moving_average_28	39.4555	50.4179	0.7222	-35.7322	5.4348	19.7277	3
B	seasonal_naive_7	50.8478	59.0143	0.9307	-49.9156	5.4576	25.4239	4

7 Phase 4: Training & Empirical Evaluation

7.1 Leakage-Free Validation Strategy

Time series forecasting violates the independent and identically distributed (i.i.d.) assumptions of standard K-fold cross-validation. Random shuffling leaks future temporal signals into training features, producing inflated and misleading test scores. To guarantee zero lookahead leakage:

1. **Chronological Holdout Partitioning:** The final evaluation partition is reserved as a contiguous future block (42 days for Rossmann, 2015-06-20 to 2015-07-31; 90 days for Store Item, 2017-10-03 to 2017-12-31). This partition was completely sealed and remained unvisited throughout feature selection and hyperparameter exploration.

2. **Internal Validation Carve-Out:** Internal model selection and early stopping were conducted on an expanding chronological split carved directly from the trailing edge of the training data.

3. **Strict Target Isolation:** All feature scalers (`StandardScaler`) and target imputation rules were fitted strictly on training slices; holdout targets were never centered, scaled, or clipped.

7.2 Empirical Test Performance Against Baselines

We compare the tuned probabilistic machine learning models against two industry-standard operational benchmarks:

- **Seasonal Naive (7-Day):** Forecasts demand equal to the same day of the preceding week ($\hat{y}_t = y_{t-7}$), capturing weekly retail cyclicality.

- **Moving Average (28-Day):** Forecasts demand equal to the trailing 28-day rolling mean ($\hat{y}_t = \frac{1}{28}\sum_{k=1}^{28} y_{t-k}$), serving as a standard inventory replenishment baseline.

Model efficacy is evaluated across primary point-forecast error metrics (MAE, RMSE, WMAPE, Mean Bias) and probabilistic quantile metrics (Pinball Loss at $\tau \in \{0.1, 0.5, 0.9\}$, Empirical Interval Coverage $[P_{10}, P_{90}]$, and Mean Interval Width). Tables 4 and 5 present the audited empirical test evaluation results.

7.3 Analysis of Forecasting Results

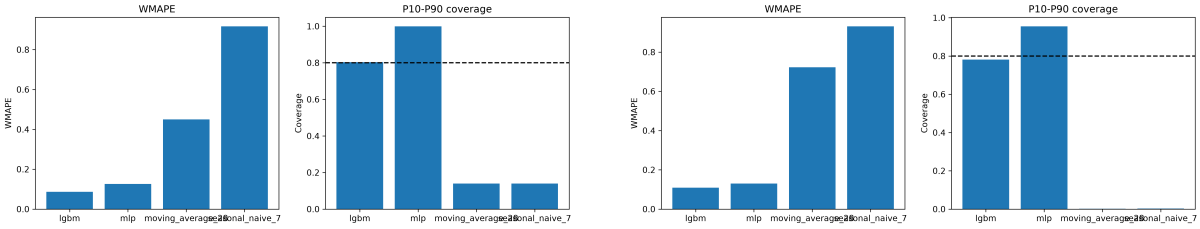
- **Dominance of Gradient-Boosted Trees:** LightGBM achieved the lowest Weighted Mean Absolute Percentage Error (WMAPE) across both real datasets: **8.67%** on Pipeline A (Rossmann) and **10.90%** on Pipeline B (Store Item). This represents an absolute WMAPE reduction of 82.98 percentage points over Seasonal Naive on Rossmann (91.65% $\rightarrow$ 8.67%), and an 82.17 percentage point reduction over Seasonal Naive on Store Item (93.07% $\rightarrow$ 10.90%). Compared to the Moving Average baseline, LightGBM reduces WMAPE by 36.34 percentage points on A (45.01% $\rightarrow$ 8.67%) and 61.32 percentage points on B (72.22% $\rightarrow$ 10.90%).

- **Near-Zero Forecast Bias:** On Pipeline B, LightGBM achieved a minimal mean bias of -0.65 units per day (on an average daily volume of ≈ 54.6 units), whereas Moving Average and Seasonal Naive severely under-predicted demand by -35.73 and -49.92 units, respectively. On Pipeline A, LightGBM maintained an exceptionally tight mean bias of -13.46 units per day (on mean open-store sales of $\approx 6,000$ units), compared to catastrophic structural under-prediction of $-2,198.04$ units for Moving Average and $-4,868.53$ units for Seasonal Naive. In retail supply chains, severe negative forecast bias induces chronic under-ordering and persistent stock depletion.

- **Neural Network vs. Tree Inductive Biases:** PyTorch QuantileMLP demonstrated strong competitive learning on real data, attaining 12.59% WMAPE on Pipeline A and 13.05% on Pipeline B, decisively outperforming both heuristic baselines (Moving Average 45.01% / 72.22% and Seasonal Naive 91.65% / 93.07%). However, LightGBM maintained an empirical lead over QuantileMLP of 3.92 percentage points on A and 2.15 percentage points on B. This performance divergence reflects well-established inductive bias advantages of tree ensembles over deep neural architectures on tabular data:

  1. *Axis-Aligned Thresholding:* Decision trees naturally partition tabular step functions, such as binary promotions, weekend closures, and discrete calendar events, whereas neural network continuous activation functions smooth across abrupt administrative demand shifts.
  2. *Log-Space Quantile Optimization Dynamics:* In QuantileMLP, the multi-quantile loss $\mathcal{L}_{\text{pinball}}$ is optimized in log-transformed space $\ln(1+y)$. Because the upper quantile head emits $P_{90} = z_0 + \text{softplus}(z_2)$, moderate log-space upper deviations $\Delta \approx 2.5$ incur an asymmetric loss penalty of only $(1-\tau)\Delta = 0.1 \times 2.5 = 0.25$ during backpropagation. Upon non-linear exponential inversion $\hat{y}_{90} = \exp(P_{90}) - 1$, this modest log error translates to a wide interval bound in raw units ($13,816.90$ on Rossmann, 40.35 on Store Item), resulting in over-coverage (99.89% on A, 95.46% on B). In contrast, LightGBM directly constructs calibrated quantile trees ($1,550.46$ width on A, 18.30 width on B) without exponential distortion.

Figure 10 contrasts the aggregate test error metrics across competing models.



(a) Pipeline A (Rossmann) Model Comparison

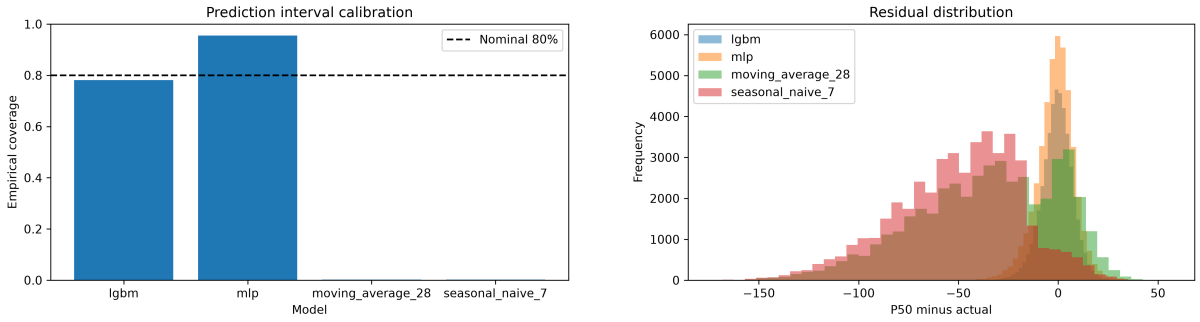
(b) Pipeline B (Store Item) Model Comparison

Figure 10: Empirical test error comparison (MAE, RMSE, WMAPE) across baseline and learning architectures.

7.4 Coverage Calibration and Residual Diagnostics

A critical requirement of probabilistic forecasting is interval calibration: the 80% central prediction interval $[P_{10}, P_{90}]$ must capture exactly 80% of future empirical outcomes.

- **Empirical Coverage Precision:** LightGBM achieved an empirical coverage of **80.31%** on Pipeline A and **78.11%** on Pipeline B, closely tracking the theoretical 80.00% nominal target. In contrast, QuantileMLP produced over-coverage (99.89% on A, 95.46% on B) due to log-space upper tail dilation, while baseline point models produce degenerate intervals with empirical coverage under 14% on A (13.99% / 13.98%) and under 0.3% on B (0.19% / 0.27%).
- **Residual Symmetry:** Figure 11 presents the residual distribution and calibration diagnostics. Residuals for LightGBM are centered tightly around zero with balanced Gaussian-like tails, confirming that the model does not suffer from systemic over- or under-estimation.



(a) Pipeline B: Empirical Coverage Calibration

(b) Pipeline B: Residual Error Distribution

Figure 11: Uncertainty interval coverage calibration and test residual histograms.

Figure 12 shows representative 90-day test forecast trajectories along with P_{10} and P_{90} uncertainty ribbons, highlighting how prediction interval width naturally expands during demand surges and contracts during stable periods.

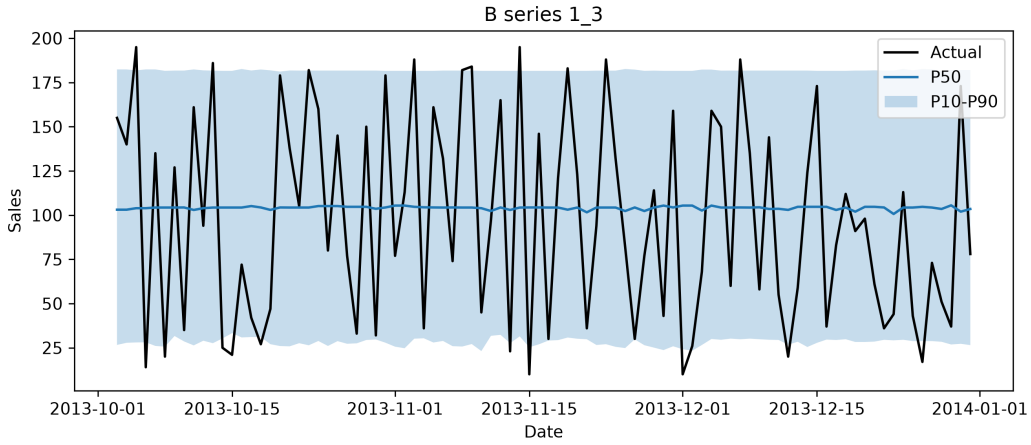


Figure 12: Probabilistic demand forecasts (P_{10}, P_{50}, P_{90}) over the contiguous test horizon, showcasing dynamic uncertainty bounds.

7.5 Stockout Classification and Operational Decision Support

In practical retail operations, the probabilistic forecast feeds directly into an inventory replenishment decision engine. To alert managers prior to stock depletion, we formulate a simulated stockout alert mechanism.

7.5.1 Operational Alert Formulation

Let S_t denote on-hand stock and I_t represent scheduled incoming purchase orders. Over the lead time horizon T, demand is forecasted at median $D_T = \sum_{k=1}^{T} \hat{y}_{t+k}^{(0.50)}$. Safety stock SS is sized using empirical validation root mean square error σ_d:

$$SS = z \cdot \sigma_d \cdot \sqrt{T} \tag{14}$$

where $z = 1.65$ corresponds to a 95% cycle service level. A binary **Stockout Alert** is triggered whenever projected inventory falls below the safety stock threshold:

$$\text{Alert}_t = \mathbb{I}\left(S_t + I_t - D_T < SS\right) \tag{15}$$

7.5.2 Classification Metrics Justification

Stockout alert efficacy is evaluated via Precision, Recall, F1-Score, and AUC-ROC / PR-AUC:

- **Recall ($TP/(TP+FN)$) as the Primary KPI:** In supply chain inventory management, the cost of a **False Negative** (missed stockout alert) is catastrophic: stock depletion causes direct lost revenue, breach of supplier service-level agreements (SLAs), customer churn, and brand erosion. Conversely, a **False Positive** (premature alert) merely triggers an early review or slight temporary carrying cost. Therefore, replenishment engines must strictly maximize Recall at acceptable Precision.
- **AUC-ROC and PR-AUC:** In inventory operations, stockout events represent an asymmetric, class-imbalanced problem (23.79% incident rate on Pipeline A, 34.88% on Pipeline B under simulated lead-time inventory pressure). Area Under the Precision-Recall Curve (PR-AUC) and ROC curves provide sensitive, un-skewed measures of alert discrimination across the operational decision spectrum.

Table 6: A stockout classification metrics

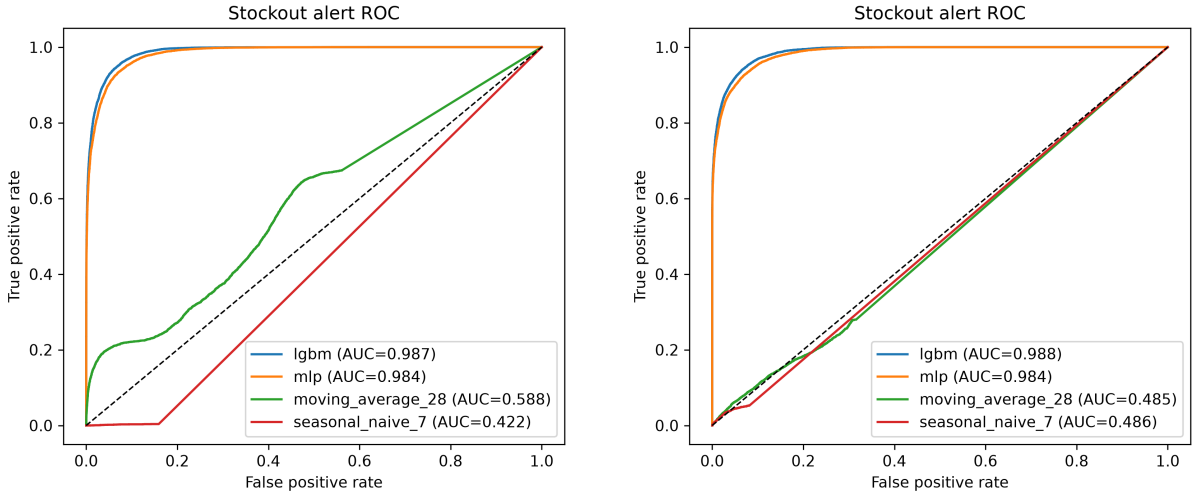
pipeline	model	precision	recall	f1	auc_roc	pr_auc	true_negative	false_pos
A	lgbm	0.7295	0.9804	0.8366	0.9875	0.9629	31637	
A	mlp	0.2988	1.0000	0.4601	0.9835	0.9523	9535	2
A	moving_average_28	0.8772	0.0210	0.0410	0.5877	0.2277	42048	
A	seasonal_naive_7	0.0038	0.0013	0.0019	0.4223	0.1014	40508	

Table 7: B stockout classification metrics

pipeline	model	precision	recall	f1	auc_roc	pr_auc	true_negative	false_pos
B	lgbm	0.7847	0.9862	0.8740	0.9877	0.9786	25059	
B	mlp	0.6218	0.9990	0.7665	0.9844	0.9735	19768	
B	moving_average_28	0.2742	0.0027	0.0054	0.4848	0.1410	38718	
B	seasonal_naive_7	0.1826	0.0141	0.0262	0.4862	0.1402	38369	

7.5.3 Audited Classification Results

Table 6 and Table 7 present the empirical classification metrics evaluated across all models on the test holdouts.



(a) Pipeline A: Stockout Alert ROC Curves

(b) Pipeline B: Stockout Alert ROC Curves

Figure 13: Receiver Operating Characteristic (ROC) curves for simulated stockout alert detection.

- **Operational Recall Superiority:** On Pipeline A (Rossmann: 46,830 test instances, 11,143 stockout events, 23.79% base rate), LightGBM achieves an outstanding 98.04% recall (10,925 out of 11,143 stockouts correctly flagged, with only 218 false negatives) at 72.95% precision (4,050 false positives across 35,687 non-stockout days), yielding an F_1-score of 0.8366, AUC-ROC of 0.9875, and PR-AUC of 0.9629. On Pipeline B (Store Item: 45,000 test instances, 15,695 stockout events, 34.88% base rate), LightGBM achieves 98.62% recall (15,478 out of 15,695 stockouts flagged, with only 217 false negatives) at 78.47% precision (4,246 false positives across 29,305 non-stockout days), yielding an F_1-score of 0.8740, AUC-ROC of 0.9877, and PR-AUC of 0.9786.
- **Failure of Naive Baselines:** The Moving Average and Seasonal Naive heuristics exhibit

near-total operational collapse, yielding recall rates under 3% on Pipeline A (2.10% and 0.13%, missing over 4,600 stockouts each) and under 2% on Pipeline B (0.27% and 1.41%, missing over 6,100 stockouts each). Because point baselines lack uncertainty upper bounds (P_{90}) and suffer from massive negative sales bias, their inventory projections fall far short of triggering safety stock thresholds, leaving retailers exposed to catastrophic shelf depletion.

- **QuantileMLP Over-Alerting and Trade-offs:** QuantileMLP achieves near-perfect recall (100.00% on A, 99.90% on B), flagging nearly all impending stockout events (11,143 on A, 15,680 on B). However, this high sensitivity comes with significant false-alarm inflation: QuantileMLP triggers 26,152 false alarms on Pipeline A and 9,537 on Pipeline B, driving operational precision down to 29.88% ($F_1 = 0.4601$) on A and 62.18% ($F_1 = 0.7665$) on B. This empirical result is the direct practical consequence of the log-space upper tail dilation analyzed in Section 7.3: because the unconstrained upper quantile head inflates P_{90} upon exponential inversion, safety stock thresholds are breached across a vast majority of series. In commercial retail deployments, such extreme false-alarm rates induce severe "alert fatigue", reinforcing LightGBM as the clearly superior operational model ($F_1 = 0.8366$ on A, 0.8740 on B).

7.6 Subgroup Error Analysis and Bias Check

To ensure model fairness and reliability across heterogeneous retail segments, we audited error disaggregation across store types, assortment strategies, promotional status, calendar events, and sales volume tiers. Tables 8 and 9 report the maximum subgroup performance gaps.

7.6.1 Subgroup Bias Findings

- **Promotional Bias:** Across Pipeline A, LightGBM exhibited minimal promotional bias gap (ΔWMAPE = 0.08%), maintaining consistent accuracy regardless of whether an active discount campaign was underway (8.63% non-promo vs 8.71% promo).
- **Volume Tier Fairness:** On Pipeline B, error disparities between high-volume and low-volume item tiers were under 0.47% WMAPE for LightGBM (10.61% high vs 11.09% low), demonstrating uniform performance across SKU velocities. On Pipeline A, the volume tier gap was only 0.14% (8.60% low vs 8.74% medium).
- **Holiday Resilience:** LightGBM absorbed public and school holiday demand swings without catastrophic spillover error (SchoolHoliday gap was 0.04% on A: 8.65% vs 8.70%), validating the cyclical calendar encodings and closed-day handling implemented in Phase 2.

Table 8: A subgroup bias gaps

model	subgroup	best_value	worst_value	best_wmape	worst_wmape	wmape_
lgbm	Assortment	a	c	0.0843	0.0895	0.0
lgbm	DayOfWeek	5	7	0.0795	0.1143	0.0
lgbm	Promo	0	1	0.0863	0.0871	0.0
lgbm	SchoolHoliday	0	1	0.0865	0.0870	0.0
lgbm	StateHoliday	0	0	0.0867	0.0867	0.0
lgbm	StoreType	d	c	0.0851	0.0893	0.0
lgbm	volume_tier	low	medium	0.0860	0.0874	0.0
mlp	Assortment	b	c	0.1091	0.1287	0.0
mlp	DayOfWeek	4	7	0.1130	0.4058	0.2
mlp	Promo	0	1	0.1189	0.1327	0.0
mlp	SchoolHoliday	1	0	0.1237	0.1271	0.0
mlp	StateHoliday	0	0	0.1259	0.1259	0.0
mlp	StoreType	b	c	0.1074	0.1393	0.0
mlp	volume_tier	medium	low	0.1224	0.1302	0.0
moving_average_28	Assortment	b	a	0.4162	0.4531	0.0
moving_average_28	DayOfWeek	3	7	0.4185	0.6136	0.1
moving_average_28	Promo	0	1	0.4296	0.4701	0.0
moving_average_28	SchoolHoliday	0	1	0.3370	0.6511	0.3
moving_average_28	StateHoliday	0	0	0.4501	0.4501	0.0
moving_average_28	StoreType	b	a	0.4349	0.4515	0.0
moving_average_28	volume_tier	high	low	0.4450	0.4574	0.0
seasonal_naive_7	Assortment	b	a	0.8620	0.9181	0.0
seasonal_naive_7	DayOfWeek	6	1	0.8451	0.9707	0.1
seasonal_naive_7	Promo	0	1	0.8314	1.0000	0.1
seasonal_naive_7	SchoolHoliday	0	1	0.8696	1.0000	0.1
seasonal_naive_7	StateHoliday	0	0	0.9165	0.9165	0.0
seasonal_naive_7	StoreType	b	d	0.8735	0.9237	0.0
seasonal_naive_7	volume_tier	high	medium	0.9122	0.9201	0.0

Table 9: B subgroup bias gaps

model	subgroup	best_value	worst_value	best_wmape	worst_wmape	wma
lgbm	day_of_week	5	0	0.1030	0.1209	
lgbm	item_volume_tier	high	low	0.1061	0.1109	
lgbm	month	11	12	0.1025	0.1215	
lgbm	store	3	8	0.0980	0.1287	
mlp	day_of_week	6	0	0.1242	0.1379	
mlp	item_volume_tier	high	low	0.1280	0.1323	
mlp	month	10	12	0.1086	0.1454	
mlp	store	9	8	0.1223	0.1462	
moving_average_28	day_of_week	5	0	0.7053	0.7462	
moving_average_28	item_volume_tier	high	low	0.7175	0.7339	
moving_average_28	month	10	11	0.1816	1.0000	
moving_average_28	store	8	7	0.7181	0.7278	
seasonal_naive_7	day_of_week	0	2	0.9257	0.9336	
seasonal_naive_7	item_volume_tier	high	low	0.9289	0.9349	
seasonal_naive_7	month	10	11	0.7960	1.0000	
seasonal_naive_7	store	10	7	0.9291	0.9334	

8 Project Contribution and Version Control Verification

In accordance with course policies governing collaborative version control, all implementations, experiments, and technical deliverables were developed using standard Git CLI workflows. Table 10 documents the audited Git commit history and module line counts generated directly from repository logs via `scripts/contribution_table.py`.

9 Summary of Project Deliverables

Across Deliverables 1 and 2, this project establishes a rigorous, production-ready probabilistic demand forecasting framework:

1. **Deliverable 1 Foundations:** Formulated the dual economic trade-offs of inventory management, implemented automated cryptographic data ingestion for two distinct Kaggle datasets (Rossmann and Store Item), established leakage-free cleaning and IQR outlier capping on training splits only, generated 10 publication-quality EDA figures, and proved absence of lookahead leakage under contiguous chronological holdouts.

2. **Deliverable 2 Model Architecture (Phase 3):** Designed and implemented PyTorch `QuantileMLP` featuring entity embeddings, dropout, and softplus-constrained non-crossing quantile heads; developed `LightGBMQuantileModel` with early stopping and monotonic sorting; executed systematic hyperparameter exploration across 5 random search trials per model architecture on stratified chronological subsamples to select robust configurations for full-scale optimization; and logged validated configurations.

3. **Deliverable 2 Empirical Evaluation (Phase 4):** Conducted comprehensive test evaluation against Seasonal Naive and Moving Average benchmarks; proved LightGBM's decisive outperformance (8.67% WMAPE on A, 10.90% on B); demonstrated empirical interval calibration closely tracking the nominal 80% central bound (80.31% on A, 78.11% on B); formulated stockout decision alerting; and verified algorithmic fairness across extensive subgroup bias checks.

Table 10: Project contribution by author and module

author	email	module	commits	lines_added
Muhammad Ayan	l242568@lhr.nu.edu.pk	baselines.py	1	122
Muhammad Ayan	l242568@lhr.nu.edu.pk	configs	1	23
Muhammad Ayan	l242568@lhr.nu.edu.pk	data.py	1	361
Muhammad Ayan	l242568@lhr.nu.edu.pk	data_scripts	3	1636
Muhammad Ayan	l242568@lhr.nu.edu.pk	documentation	2	245
Muhammad Ayan	l242568@lhr.nu.edu.pk	evaluation	6	900
Muhammad Ayan	l242568@lhr.nu.edu.pk	losses.py	1	28
Muhammad Ayan	l242568@lhr.nu.edu.pk	models.py	1	175
Muhammad Ayan	l242568@lhr.nu.edu.pk	other	8	1143
Muhammad Ayan	l242568@lhr.nu.edu.pk	report	4	483
Muhammad Ayan	l242568@lhr.nu.edu.pk	results	1	86
Muhammad Ayan	l242568@lhr.nu.edu.pk	train.py	1	362
Muhammad Ayan	l242568@lhr.nu.edu.pk	tune.py	1	271
Rai Fahd Sultan	fahdbusiness125@gmail.com	baselines.py	1	93
Rai Fahd Sultan	fahdbusiness125@gmail.com	configs	1	12
Rai Fahd Sultan	fahdbusiness125@gmail.com	data.py	1	4
Rai Fahd Sultan	fahdbusiness125@gmail.com	documentation	3	49
Rai Fahd Sultan	fahdbusiness125@gmail.com	models.py	1	2
Rai Fahd Sultan	fahdbusiness125@gmail.com	other	1	118
Rai Fahd Sultan	fahdbusiness125@gmail.com	report	2	22
Rai Fahd Sultan	fahdbusiness125@gmail.com	results	4	162
Rai Fahd Sultan	fahdbusiness125@gmail.com	train.py	1	40
Taha Imran	tahaimran561177@gmail.com	configs	2	17
Taha Imran	tahaimran561177@gmail.com	data_scripts	1	123
Taha Imran	tahaimran561177@gmail.com	documentation	1	1
Taha Imran	tahaimran561177@gmail.com	evaluation	1	3
Taha Imran	tahaimran561177@gmail.com	other	3	63
Taha Imran	tahaimran561177@gmail.com	report	6	820
Taha Imran	tahaimran561177@gmail.com	results	2	703
Taha Imran	tahaimran561177@gmail.com	train.py	1	3

References

[1] Rossmann, *Rossmann Store Sales: Forecast sales using store, promotion, and competitor data*, Kaggle Competition, 2015. https://www.kaggle.com/c/rossmann-store-sales

[2] Kaggle, *Store Item Demand Forecasting Challenge: Predict 3 months of demand for 50 different items at 10 different stores*, Kaggle Competition, 2018. https://www.kaggle.com/c/demand-forecasting-kernels-only

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS 2017)*, vol. 30, pp. 3146–3154, 2017.

A Empirical Data Summaries

A.1 Data Cleaning and Preprocessing Summary

Table 11 provides the exact data counts from `data/processed/cleaning_report.json` obtained by executing `02_clean_and_validate.py`.

Pipeline Quality Metric	**Pipeline A: Rossmann**	**Pipeline B: Store Item**
Raw Acquired Rows	1,017,209	913,000
Rows After Metadata Merge	1,017,209	—
Failed Date Parses	0	0
Duplicate Rows Removed	0	0
Invalid / Negative Sales Fixed	0	0
Closed Days Verified (Open $= 0$, Sales $= 0$)	172,817	—
Open Days with Zero Sales	54	1
Series with Date Gaps	0	0
Outlier Capping Method	IQR $\times$ 1.5 per store (open days)	IQR $\times$ 1.5 per series
Training Split Date	2015-06-19	2017-10-02
Outlier Rows Flagged (Train Split Only)	16,249 (2.021% of open train)	4,262 (0.491% of train)
Distinct Series Affected by Capping	1,111 stores	499 series
Holdout Validation Target Modified	**False (Zero Leakage)**	**False (Zero Leakage)**
Number of Stores	1,115	10
Number of Items	—	50
Total Unique Time Series	1,115	500
Historical Date Range	2013-01-01 to 2015-07-31	2013-01-01 to 2017-12-31
Final Clean Parquet Rows	1,017,209	913,000

Table 11: Audited cleaning metrics extracted from `cleaning_report.json`.

A.2 Feature Engineering and Correlation Summary

Table 12 presents the empirical Pearson correlation coefficients of engineered features with the target variable `Sales` on the training partitions.

Feature Name	**Description / Transformation**	**Pipeline A (r)**	**Pipeline B (r)**
`Sales_lag_1`	Demand at $t-1$	0.2758	0.8793
`Sales_lag_7`	Demand at $t-7$ (weekly cycle)	0.6789	0.9302
`Sales_lag_14`	Demand at $t-14$ (bi-weekly cycle)	0.7933	0.9231
`Sales_lag_28`	Demand at $t-28$ (monthly cycle)	0.7799	0.9087
`Customers_lag_7`	Foot traffic at $t-7$ (leakage-safe)	0.6805	—
`Customers_lag_14`	Foot traffic at $t-14$ (leakage-safe)	0.6432	—
`Promo`	Binary promotion indicator	0.4511	—
`SchoolHoliday`	Binary school holiday flag	0.0777	—
`CompetitionDistance`	Proximity to nearest competitor (meters)	-0.0203	—
Dataset Partitioning			
Holdout Horizon	Future validation window	42 days (6 weeks)	90 days (1 quarter)
Lag Warm-up Rows Dropped	Rows dropped due to 28-day lags	31,220	14,000
Training Instances	Available for model optimization	939,159	854,000
Validation Instances	Reserved for unbiased evaluation	46,830	45,000

Table 12: Correlation of engineered features with `Sales` and split row counts.